Introduction to Microcontrollers

Facilitator Guide

Prepared by BLIS GLOBAL Robotics Department

Copyright @2024

WEEK7: Advanced Arduino Programming

1. Libraries and Shields

In Arduino programming, **libraries** and **shields** make working with external components easier and more efficient. They allow you to expand the capabilities of your Arduino without writing complex code from scratch.

Arduino Libraries:

A library is a collection of pre-written code that simplifies tasks for interfacing with components such as sensors, actuators, or communication devices. Libraries abstract the low-level details and allow you to focus on the core functionality.

- **How to Use a Library:**
    1. **Including a Library**: Libraries are included at the start of your Arduino program using the #include directive.
    2. **Accessing Library Functions**: Once included, you can call the functions provided by the library to interact with components.

Popular Arduino Libraries:

1. **Servo Library**: Controls servo motors.

```cpp
#include <Servo.h>
Servo myServo;
```

2. **Wire Library**: Used for I2C communication with other devices like sensors.

```cpp
#include <Wire.h>
```

1. **Adafruit Sensor Libraries**: Includes libraries for sensors like temperature and motion.

- **Installing Libraries**: You can install libraries via the Arduino IDE by navigating to **Sketch > Include Library > Manage Libraries**.

Arduino Shields:

Shields are circuit boards that plug on top of your Arduino and provide specific functions, such as motor control, wireless communication, or sensor interfaces. They help simplify wiring and reduce the complexity of connecting multiple components.

Popular Shields:

1. **Motor Shield**: Controls motors, allowing your Arduino to drive DC motors or stepper motors.
2. **Sensor Shield**: Makes it easier to connect various sensors to your Arduino.
3. **Wi-Fi Shield**: Adds wireless internet connectivity to your Arduino projects.

Example: The **L298N Motor Shield** allows you to control the speed and direction of DC motors, and it works in conjunction with the AFMotor library for easy programming.

---

2. Communication Protocols

When your Arduino needs to communicate with other devices, it uses various communication protocols. The three most commonly used protocols are **I2C**, **SPI**, and **UART**.

I2C (Inter-Integrated Circuit):

- **Overview**: I2C is a protocol that allows communication between multiple devices using just two wires: **SDA (Data)** and **SCL (Clock)**.
- **Usage**: It's commonly used to connect sensors, LCD displays, and real-time clocks (e.g., DS3231).
- **Master-Slave Relationship**: One device (the master, usually the Arduino) controls the clock and manages communication with multiple slaves.

Example Code: Using I2C with the Wire Library

```cpp
#include <Wire.h>

void setup() {
  Wire.begin();  // Start I2C communication as master
}

void loop() {
  Wire.beginTransmission(0x27); // Start communication with a device at address 0x27
  Wire.write("Hello");          // Send data
  Wire.endTransmission();       // End communication
  delay(1000);
}
```

SPI (Serial Peripheral Interface):

- **Overview**: SPI is a high-speed protocol used for communication between microcontrollers and peripherals (like SD cards and sensors). It uses four main signals: **MOSI (Master Out, Slave In)**, **MISO (Master In, Slave Out)**, **SCK (Clock)**, and **SS (Slave Select)**.
- **Usage**: SPI is commonly used when speed is important, such as in memory modules or displays.

UART (Universal Asynchronous Receiver-Transmitter):

- **Overview**: UART is the most basic form of communication between two devices. It requires two lines, **TX (Transmit)** and **RX (Receive)**, to communicate serially.
- **Usage**: UART is used in serial communication between an Arduino and devices like Bluetooth modules, GSM modules, or between two Arduinos.

Example Code: Communication Between Two Arduinos Using UART

```cpp
void setup() {
  Serial.begin(9600);  // Initialize serial communication at 9600 baud rate
}

void loop() {
  Serial.println("Hello from Arduino!");  // Send data to the other device
  delay(1000);
}
```

---

3. Hands-On Activity: Use a Library to Interface with a Sensor

In this activity, we will use the **DHT11 temperature and humidity sensor** with the Arduino. The sensor will measure temperature and humidity, and we will display the values on the serial monitor using the **DHT library**.

Materials Needed:

- Arduino Uno
- DHT11 Sensor
- Jumper wires
- Breadboard

Connections:

1. **DHT11 Pin 1 (VCC)**: Connect to the 5V pin on Arduino.
2. **DHT11 Pin 2 (Data)**: Connect to digital pin 2 on Arduino.
3. **DHT11 Pin 4 (GND)**: Connect to the GND pin on Arduino.

Code: Using the DHT Library to Read Data

```cpp
#include <DHT.h>

#define DHTPIN 2        // DHT11 is connected to digital pin 2
#define DHTTYPE DHT11   // DHT 11 sensor type

DHT dht(DHTPIN, DHTTYPE);  // Initialize DHT object

void setup() {
  Serial.begin(9600);
  dht.begin();  // Start the DHT sensor
}

void loop() {
  float humidity = dht.readHumidity();      // Read humidity
  float temperature = dht.readTemperature(); // Read temperature

  // Display results
  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.print(" %\t");
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" *C");

  delay(2000); // Wait 2 seconds before reading again
}
```

Explanation:

- The DHT.h library is included to easily interface with the sensor.
- The dht.readHumidity() and dht.readTemperature() functions are used to obtain the sensor readings.

- The values are printed to the serial monitor.

---

4. Hands-On Activity: Communication Between Two Arduino Boards

In this activity, we will implement communication between two Arduino boards using the **I2C protocol**. One Arduino will act as the master, and the other as the slave. The master will send a message to the slave, and the slave will respond.

Materials Needed:

- Two Arduino boards (e.g., Arduino Uno)
- Jumper wires
- Breadboard

Connections:

1. Connect the **SDA** and **SCL** pins of both Arduino boards together.
2. Connect the **GND** of both Arduinos together.

Master Code: Sending Data to Slave

cpp Copy code

```cpp
#include <Wire.h>

void setup() {
  Wire.begin();  // Initialize I2C communication as master
  Serial.begin(9600);
}

void loop() {
  Wire.beginTransmission(8);  // Start communication with slave at address 8
  Wire.write("Hello, Slave"); // Send message
  Wire.endTransmission();     // End communication
  delay(1000);
}
```

Slave Code: Receiving Data from Master

cpp Copy code

```cpp
#include <Wire.h>

void setup() {
  Wire.begin(8);  // Initialize I2C communication with address 8
  Wire.onReceive(receiveEvent);  // Register event handler for receiving data
  Serial.begin(9600);
}

void receiveEvent(int howMany) {
  while (Wire.available()) {
    char c = Wire.read();  // Read data from master
    Serial.print(c);       // Print data to serial monitor
  }
  Serial.println();        // Move to the next line
}

void loop() {
  // Nothing in the loop; data is handled in receiveEvent()
}
```

Explanation:

- The **master** sends a string message ("Hello, Slave") to the **slave** over the I2C bus.
- The **slave** listens for incoming data and prints it to the serial monitor.

This activity demonstrates how Arduino boards can communicate using the I2C protocol. This concept can be extended to interface multiple sensors or devices in a larger system.

Summary:

- **Libraries** provide pre-written functions to simplify code for components like sensors or motors.
- **Shields** expand Arduino's functionality by adding hardware capabilities such as motor control or wireless communication.
- **Communication protocols** like **I2C**, **SPI**, and **UART** allow Arduino to interface with sensors, actuators, and other microcontrollers.
- Through hands-on activities, you learned to interface with a sensor using a library and to implement communication between two Arduino boards using I2C.

By mastering libraries, shields, and communication protocols, you'll be able to build more complex and functional Arduino projects that integrate multiple devices.